极客时间

多 Agent 设计模式

工程思想：上下文窗口隔离

第 3 周 · 第 9 节

讲师：黄佳

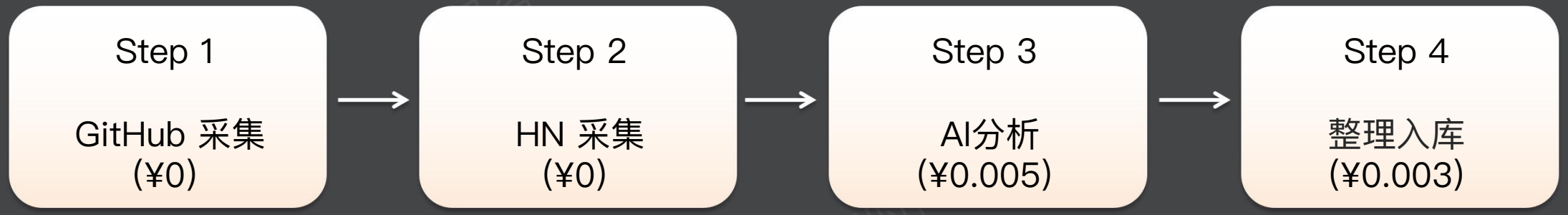
极客时间黄佳老师出品

极客时间

V2 回顾 — 线性流水线

极客时间

pipeline.py (V2 线性流水线)



三大局限:

① 串行瓶颈： 一步卡住全线停

② 职责混杂 — 一个 prompt 分析所有类型

③ 无审核 — 分析完直接入库

极客时间黄佳老师出品

V2 → V3 升级目标

极客时间

V2（单兵作战）

单一分析 prompt
所有内容同一套处理逻辑

串行流水线
一步卡住全线停

无审核机制
分析完直接入库

月成本 ¥5–10

V3（团队协作）

专业化 Agent
论文、产品、博客各有专家

并行分析 + 汇总
三个 Agent 同时工作

Supervisor 审核循环
不合格打回修改

月成本 ¥15–25（质量大幅提升）

极客时间

本周工程思想

上下文窗口隔离
Sub-agent 的核心价值
不是“更强的能力”而是“更干净的环境”

Context isolation, not capability multiplication

量化对比 — 单 Agent vs 多 Agent 的上下文

极客时间

单 Agent（开放办公室）

处理 10 个任务
每个任务的上下文带着前面所有任务

第 1 个任务：5K tokens
第 5 个任务：25K tokens
第 10 个任务：50K tokens

噪声逐步累积
后面的任务质量不如前面
成本也越来越高

10 个 Sub-agent（独立会议室）

各自处理 1 个任务
每个 Agent 只有自己任务的上下文

Agent 1：5K tokens
Agent 5：5K tokens
Agent 10：5K tokens

始终干净
每个任务质量一致
成本可预测

工程思想卡 — 上下文窗口隔离

极客时间

一句话： 上下文越干净，输出越准确

为什么重要：

- 噪声成本是隐形的——你看不到它
- 但它在悄悄降低每一次输出的质量
- 而且你为那些噪声 token 付了真金白银

本节验证：

- 5 种多 Agent 设计模式，都在用不同方式实现隔离
- Router 隔离不同类型的处理逻辑
- Orchestrator 隔离规划与执行
- Supervisor 隔离产出与审核

通用原则（超越 AI）：

- 在任何系统中，减少无关信息对决策单元的干扰
- 都能提升决策质量
- 微服务 vs 单体架构，模块化 vs 大泥球——同一个道理

极客时间

极客时间

五种多 Agent 设计模式 — 总览

极客时间

模式	核心动作	类比	Agent 数量	适用场景
Router	分发	医院分诊台	1 路由 + N 处理	输入类型多样
Orchestrator	拆解+汇总	乐队指挥	1 编排 + N 执行	复杂任务需拆解
Divide & Conquer	切块+合并	大扫除	N 同构 Agent	大数据量同构处理
Competition	竞争+择优	广告比稿	N 竞争 + 1 评审	高风险决策
Supervisor	执行+审核	编辑审稿	1 执行 + 1 审核	质量要求高

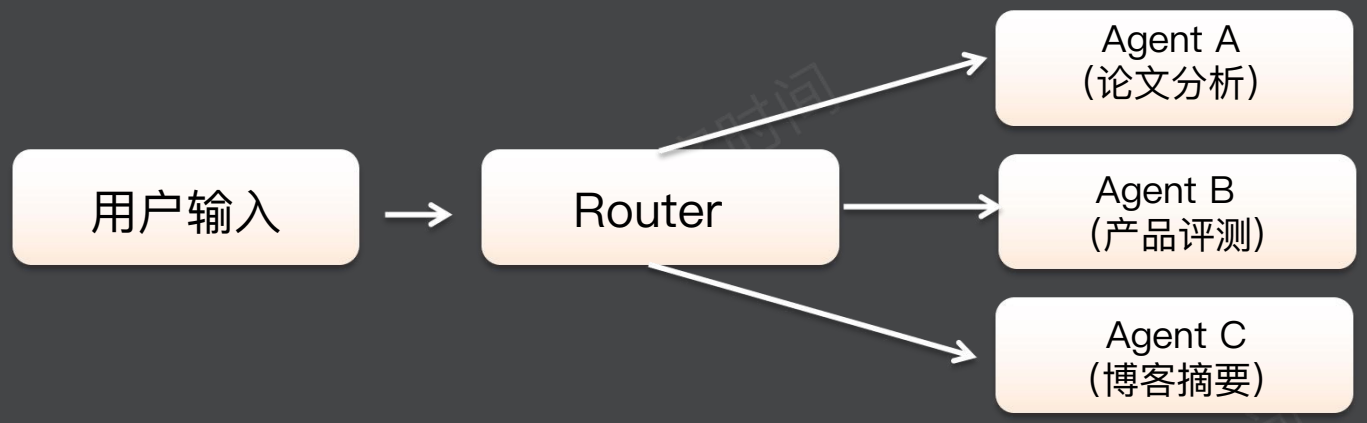
Pattern Selection Card: ASSESS → ROUTE → SELECT

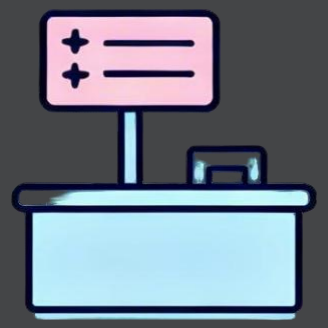
极客时间

步骤	做什么	关键问题
ASSESS	评估任务特征	任务复杂度？数据量？质量要求？
ROUTE	确定协作强度	需要几个 Agent？它们如何交互？
SELECT	选定具体模式	Router / Orchestrator / D&C / Competition / Supervisor？

极客时间黄佳老师出品

模式 1 Router（路由模式）： 医院分诊台





核心：根据输入内容，分发到不同的处理 Agent
类比：医院分诊台 — 护士根据症状分到不同科室

关键决策：
- 路由判断要快（低温度 + 短输出）
- 分类错误的代价 > 分类调用的成本
- 可以用规则路由替代 LLM 路由（如果特征明显）

极客时间黄佳老师出品

Router 代码示例

极客时间

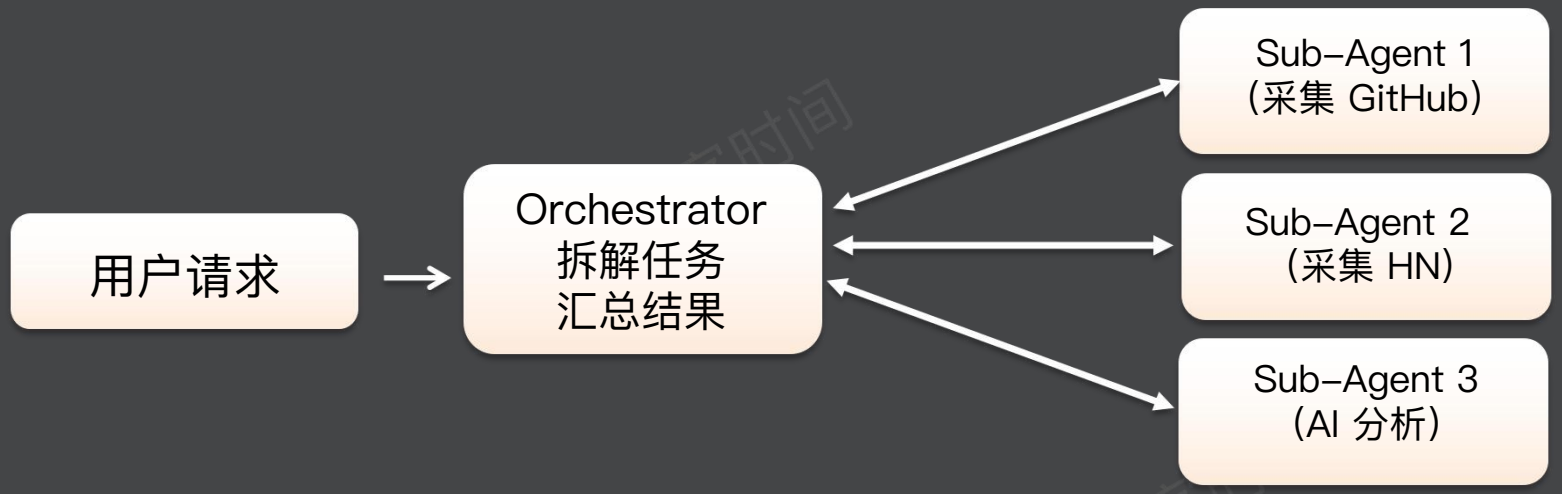
project/ai-knowledge-base/agents/router.py | python

```python
from model_client import chat

def router(content: str) -> str:
    """根据内容类型路由到不同分析 Agent"""
    classify_prompt = (
        "请判断以下内容的类型，只回复一个词："
        "paper / product / blog\n\n"
        f"内容：{content[:500]}"
    )
    content_type = chat(classify_prompt,
                        temperature=0.1).strip().lower()

    prompts = {
        "paper": "你是学术论文分析专家。提取研究问题、方法、结论、创新点。",
        "product": "你是技术产品评测专家。分析核心功能、技术栈、适用场景。",
        "blog": "你是技术博客编辑。总结核心观点、关键论据、实用建议。",
    }
    system = prompts.get(content_type, prompts['blog'])
    return chat(content, system=system)
```

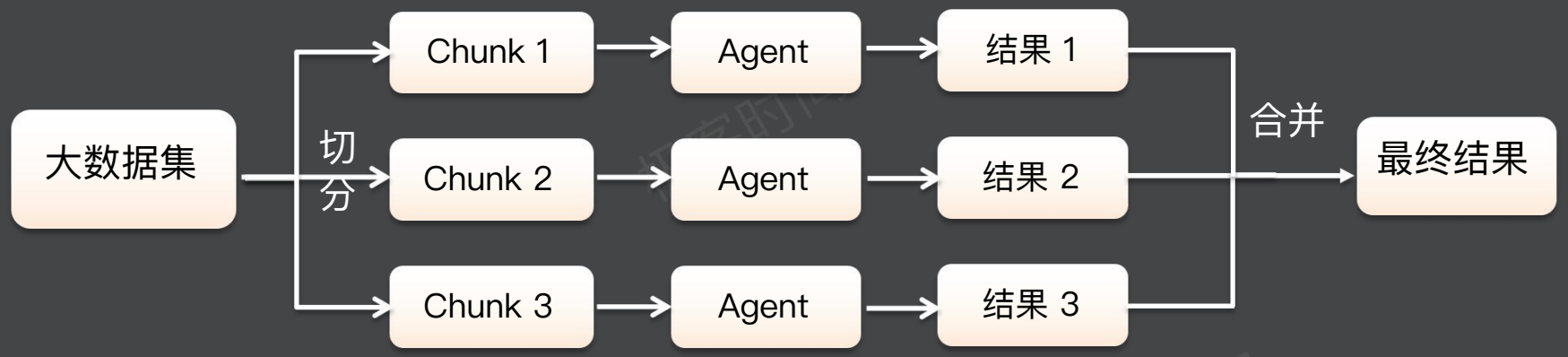
模式 2 Orchestrator（编排模式）： 乐队指挥



核心：一个中心 Agent 拆解任务、分配子任务、汇总结果
类比：乐队指挥 — 不自己演奏，协调各声部合成交响乐

关键决策：
- Orchestrator 不做具体分析（只规划 + 汇总）
- 子任务之间尽量无依赖（可并行）
- 汇总时消除重复、突出共识

模式 3 Divide-and-Conquer（分治模式）：大扫除



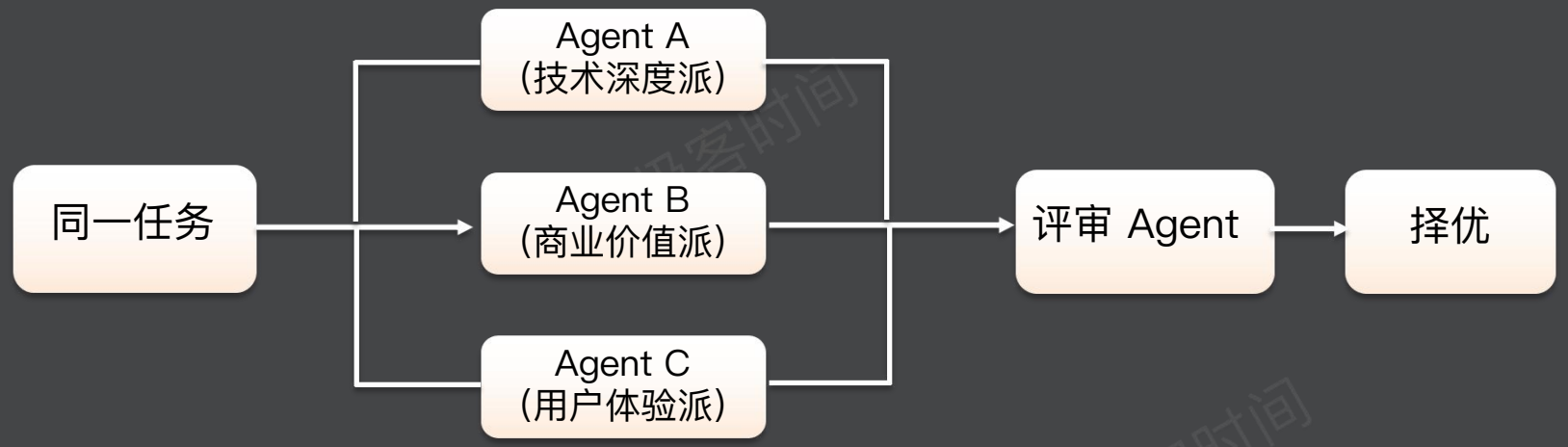
核心：把大任务切成同构小块，各自独立处理，最后合并
类比：大扫除 — 客厅/卧室/厨房各一人，最后汇合

关键决策：
- chunk 大小的权衡（太小=调用多，太大=上下文脏）
- 建议：5–10 条/chunk

与 Orchestrator 的区别：
- Orchestrator：子任务是异构的（不同类型）
- D&C：子任务是同构的（同一处理逻辑，不同数据）

极客时间黄佳老师出品

模式 4：Competition（竞争模式）：广告比稿

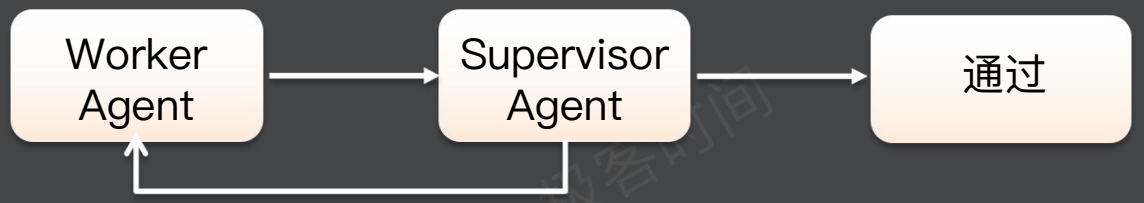


核心：多个 Agent 对同一任务给出各自方案，最终择优
类比：广告比稿 — 三家公司各出方案，甲方选最好的

关键决策：
- 竞争方案的差异要够大（不同 system prompt + 高温度）
- 评审 Agent 需要明确的评分标准
- 成本是其他模式的 N 倍 — 只用于高价值决策

极客时间黄佳老师出品

模式 5：Supervisor（监督模式）：编辑审稿



不通过（附带反馈） 最多三轮

核心：一个 Agent 执行，另一个 Agent 审核
类比：编辑审稿 — 作者写稿，编辑审稿，不合格打回修改

审核循环的三要素：
① Worker 产出初稿
② Supervisor 审核 + 给出具体反馈
③ Worker 根据反馈修改（不是重做，是改进）

关键决策：
- max_retries = 3（再多说明 prompt 有问题）
- 反馈要具体（不能只说「不好」，要说「哪里不好」）

极客时间黄佳老师出品

Supervisor 代码示例

极客时间

project/ai-knowledge-base/agents/supervisor.py | python

```python
from model_client import chat
import json

def supervisor(task: str, max_retries: int = 3) -> dict:
    worker_sys = '你是 AI 分析师。输出 JSON 格式。'
    review_sys = ('审核分析报告。输出 JSON: '
                  '{"passed": bool, "score": 1-10, "feedback": "..."}')

    output, feedback = None, None
    for attempt in range(1, max_retries + 1):
        # Worker 执行（首次直接做，后续带反馈改进）
        prompt = task if attempt == 1 else (
            f'任务: {task}\n上次产出: {output}\n'
            f'反馈: {feedback}\n请改进。')
        output = chat(prompt, system=worker_sys)
```

project/ai-knowledge-base/agents/supervisor.py | python

```python
        # Supervisor 审核
        review = chat(f'审核:\n{output}',
                      system=review_sys, temperature=0.2)
        data = json.loads(review)
        print(f'  第 {attempt} 轮: {data["score"]}/10')

        if data.get('passed'):
            return {'output': output, 'attempts': attempt}
        feedback = data.get('feedback', '请改进')

    return {'output': output, 'attempts': max_retries,
            'warning': '达到最大重试次数'}
```

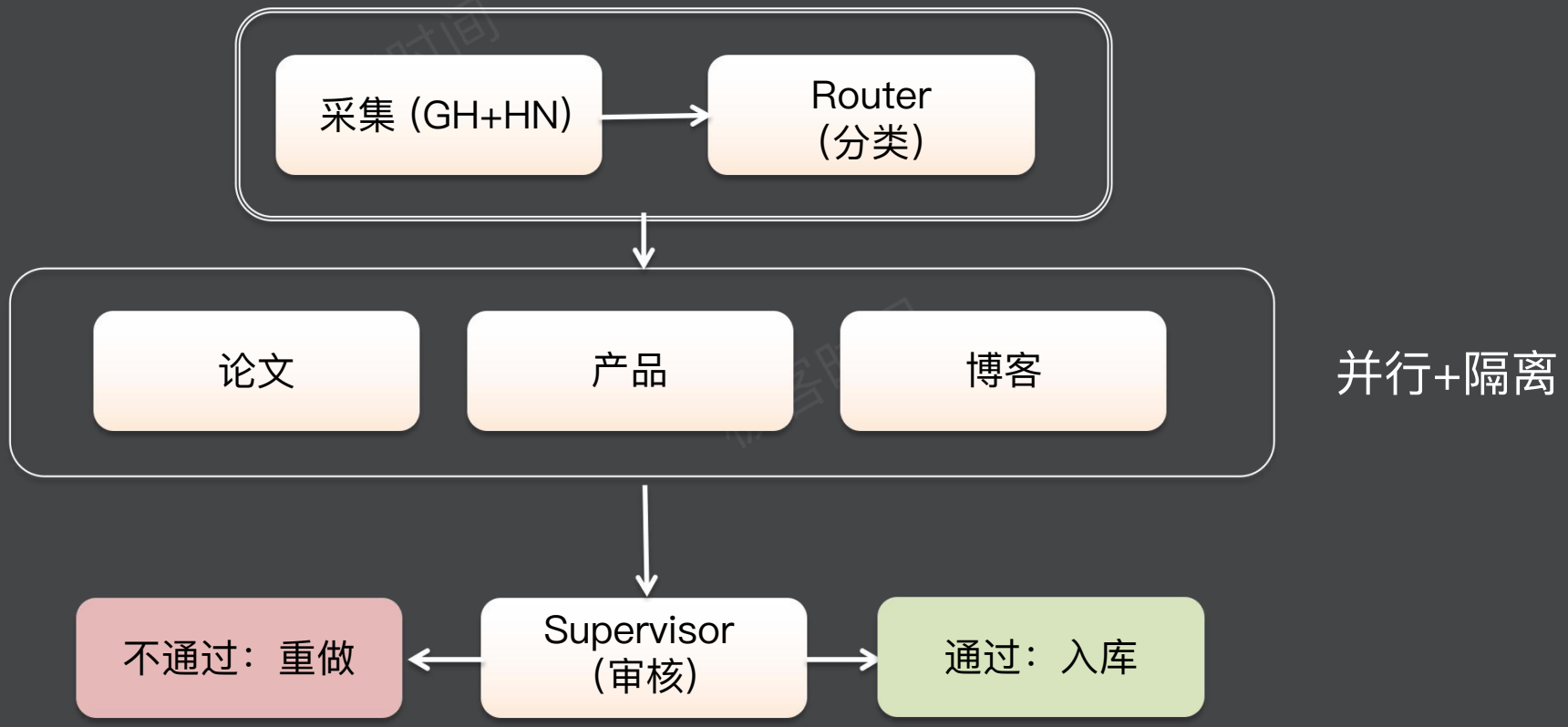
V3 选型分析 — 为什么选 Router + Supervisor?

极客时间

V2 局限	具体表现	V3 解法	对应模式
职责混杂	一个 prompt 分析所有类型	专业化 Agent 分工	Router
串行瓶颈	分析和整理顺序执行	并行分析 + 异步汇总	Orchestrator
无质量保障	分析完直接入库	审核循环（最多3轮）	Supervisor

极客时间黄佳老师出品

V3 架构图 — 多 Agent 协作



极客时间黄佳老师出品

V3 代码结构 — 需要新增的文件

main ai-knowledge-base / v3-multi-agent / patterns /

huangjia2019 feat: complete course template - V1 through V4

Name	Last commit message
..	
__init__.py	feat: complete course t
router.py	feat: complete course t
supervisor.py	feat: complete course t

main ai-knowledge-base / v3-multi-agent /

..	
.github/workflows	feat:
.opencode	feat:
hooks	feat:
knowledge	feat:
patterns	feat:
pipeline	feat:
tests	feat:
workflows	feat:
.env.example	feat:
AGENTS.md	feat:
requirements.txt	feat:

极客时间黄佳老师出品

V2 vs V3 成本对比

极客时间

步骤	V2 成本	V3 成本	差异原因
采集	¥0	¥0	不变（纯 API）
路由分类	—	¥0.001	新增 Router
分析	¥0.005	¥0.01	3 个专业 Agent
汇总	—	¥0.003	新增汇总 Agent
审核	—	¥0.005	新增 Supervisor
重做（平均）	—	¥0.005	约 30% 需要 1 轮重做
合计/次	¥0.008	¥0.024	约 3 倍
月度估算	¥5–10	¥15–25	质量大幅提升

本节作业

极客时间

任务 1：实操 1：实现 Router 路由模式

创建 agents/router.py，实现内容分类功能。输入一段技术内容，输出分类结果（paper/product/blog），根据分类调用不同的分析 prompt。用 3 段不同类型的内容测试分类准确率。

任务 2：实操 2：实现 Supervisor 监督模式

创建 agents/supervisor.py，实现审核循环。Worker 产出分析报告，Supervisor 审核评分。不通过则带反馈重做，最多 3 轮。验证：观察分数是否随轮次递增。

极客时间

本节小结

多 Agent 本质是上下文隔离——不是更强是更干净

用ASSESS-ROUTE-SELECT 选模式

V3 = Router + Supervisor

极客时间

下一节 第 10 节 LangGraph 状态机 — 用代码实现 V3 架构